

System Design and Implementation of a 2D Side-Scrolling Action Game

Yi-Ting Meng, Wei-ting Liu

Dept. of mathematics

National Central University

Taoyuan, Taiwan

Student ID: 112201535, 112201040

Abstract—This paper presents the system design and implementation of a 2D side-scrolling action game developed utilizing the Unity engine and C# programming language. The research focuses on the player control system, environmental interaction logic, and high-complexity combat mechanics. To enhance the dynamic challenge of the gameplay, the Boss AI is constructed using a Finite State Machine (FSM) architecture, facilitating diverse behavioral patterns including long-range magic casting, melee attacks, and teleportation. Addressing critical technical challenges encountered during implementation—specifically Object Lifecycle Management during player respawn and the “Boss Behavior Freeze” anomaly—this study proposes an asynchronous reset mechanism integrating the Singleton Pattern with Coroutines. Experimental results demonstrate that this mechanism effectively resolves state synchronization conflicts, thereby ensuring the stability of the Game Loop and the robustness of the system during scene resets. A demonstration video of the gameplay and system implementation is available at <https://youtu.be/KWVxwBmTp5U>.

Index Terms—Unity, C#, Finite State Machine (FSM), Game Loop, Object-Oriented Programming, Coroutine

I. INTRODUCTION

A. Motivation and Objective

The primary objective of this project is to leverage the Unity engine and the C# programming language to construct a 2D action platformer featuring a complete Game Loop. The core research focus lies in validating the application of **Object-Oriented Programming (OOP)** principles within complex game systems. Through modular design, this study implements Boss AI logic, scene interaction mechanisms (e.g., interactive props and save points), and global state management (victory/defeat determination), thereby ensuring the scalability and maintainability of the system.

B. System Scope and Gameplay Mechanics

The core gameplay of the proposed system revolves around a **physics-oriented character controller**. Players are required to manipulate the protagonist to execute composite commands—such as movement, jumping, and attacking—to navigate through dynamic environmental traps and counteract hostile units. The system scope is defined by the following three key components:

- **Core Mechanics:** Implementation of precise collision detection and physical feedback to ensure the fluidity of operational control.

- **Progression System:** Integration of key collection and Checkpoint mechanisms to facilitate non-linear level progression logic.
- **Boss Encounter:** The endgame is designed as a combat encounter with the “Dark Fairy.” This Boss features multi-stage attack patterns driven by a **Finite State Machine (FSM)**, designed to rigorously test the player’s mastery of the system mechanics.

II. SYSTEM ARCHITECTURE

To ensure the consistency and robustness of the runtime logic, the execution flow of the system is designed as illustrated in Fig. 1.

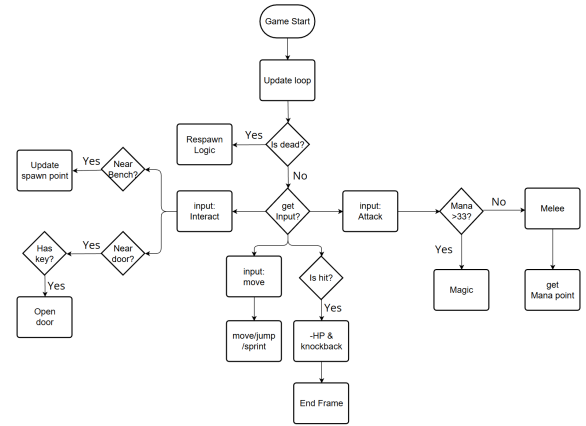


Fig. 1. System Execution Flowchart.

A. Priority Logic

The execution sequence within the Unity Update() lifecycle adopts a “**Liveness-First**” strategy. The primary step involves evaluating the Is Dead? condition. This prioritization effectively preempts logical anomalies caused by “**Post-mortem Actions**” (e.g., triggering attack events after Health Points reach zero), thereby enforcing a strict state validity check.

B. Input and State Branching

Upon confirmation of the liveness status, the **Input Processing** module distributes control among three major subsystems:

- **Locomotion:** Manages fundamental movement and physics-based jumping mechanics.
- **Interaction:** Detects trigger events associated with environmental entities (e.g., Treasure Chests, Save Points).
- **Combat Logic:** Dynamically branches into Melee or Ranged Magic attacks based on specific resource thresholds (e.g., when *Mana* > 33).

This architectural design establishes a closed-loop system characterized by high **Determinism**, ensuring predictable behavior for every frame.

III. GAME MECHANICS AND SYSTEM DESIGN

This section elucidates the architectural design of the core gameplay systems, encompassing the **Kinematic Mechanics** of the player character and the behavioral decision-making logic of hostile units. Implemented leveraging the Unity physics engine and C# scripting, these mechanisms aim to strike a strategic balance between operational fluidity and combat challenge.

A. Player Kinematics and Control Mechanisms

The **Player Controller** adopts a **Physics-driven** architecture. By real-time monitoring of the **Input System** and applying forces or modifying velocity vectors on the **Rigidbody2D**, the system achieves precise control characterized by inertia and weightiness. The implementation details of the core mechanisms are as follows (See Fig. 2):

1) *Locomotion (Physics-based Movement)*: Movement logic involves manipulating the **rigid body velocity** rather than merely modifying **Transform** coordinates. By calculating horizontal vectors via **W/A/S/D** inputs and fine-tuning the friction parameters of the **Physics Material 2D**, the system realizes a smooth **Acceleration/Deceleration Curve**, ensuring tight handling without the sensation of “slipping”.

2) *Double Jump (State Evaluation)*: The system maintains a jump counter (`jumpCount`) and a ground detection boolean (`IsGrounded`). Upon detecting a **Space** key input, if the character is airborne and `jumpCount` > 0, the system resets the vertical velocity and applies an instantaneous upward force. This allows for secondary vertical displacement, significantly expanding the exploration capability within vertical terrain.

3) *Dash and Temporal Invincibility*: The dash mechanism is managed by a **Coroutine**. When triggered, the system locks the character’s velocity to the dash vector and temporarily disables the **Hurtbox** or switches the character’s layer to an **Invincible Layer**. This **“i-frames”** mechanism is utilized not only for rapid displacement but also provides tactical options for traversing through enemy barrages and obstacles.

4) *Downward Strike / Pogo (Recoil Mechanics)*: Referencing the classic mechanism from “*Hollow Knight*,” when a “Down + Attack” command is detected and the attack **Hitbox** collides with an enemy or environmental trap, a **Recoil Feedback** is triggered. The program forcibly resets the player’s vertical velocity to zero and applies an upward

Impulse Force. This mechanism transforms offensive actions into advanced traversal techniques, allowing players to utilize enemies as dynamic platforms.

5) *Damage Knockback (Vector Calculation)*: To prevent the player from falling into a “**Stunlock**” loop (continuous hitstun), a damage repulsion mechanism is implemented. Upon the invocation of `TakeDamage()`, the system calculates the knockback vector based on the relative position of the damage source:

$$V_{knockback} = (P_{player} - P_{source})_{normalized} \times F_{force} \quad (1)$$

Subsequently, an instantaneous force (`ForceMode2D.Impulse`) is applied to the player using $V_{knockback}$. This mechanism not only provides visual and physical feedback but also forcibly separates the player from the hazardous **Hitbox**, thereby ensuring gameplay fairness and error tolerance.

B. Enemy AI Behavior and Variety

To enrich the level experience and combat pacing, this study designed four distinct hostile units possessing different behavioral logic structures (See Fig. 3):

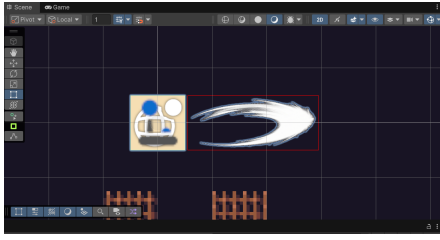
1) *Ground Patroller (Edge Detection AI)*: Utilizes **Raycast** technology to detect platform edges and obstacles. When the **Raycast** fails to hit the ground or intersects a wall, the AI automatically inverts its movement vector. Furthermore, the Z-axis rotation of the **Rigidbody2D** is locked (**Freeze Rotation**) to prevent unintended tumbling caused by physics calculations.

2) *Self-Destructing Crawler (Proximity FSM)*: This unit possesses distance-based state switching capabilities. Calculating the proximity to the player in real-time via `Vector2.Distance`:

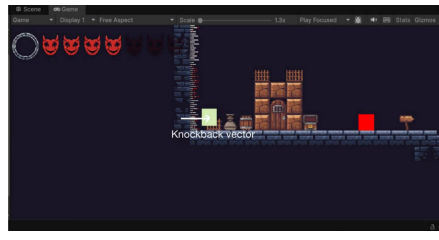
- **Chase:** Accelerates tracking when entering the alert range.
- **Detonate:** Triggers a “Detonation Sequence” upon entering the blast radius, executing a color-flashing warning via a **Coroutine** and ultimately inflicting **Area of Effect (AoE) Damage**.

3) *Ranged Flying Unit (Projectile System)*: Suspended in mid-air, this unit maintains a fixed distance (`Keep Distance`) from the player, introducing a vertical threat. The AI periodically invokes the `Instantiate` method to generate **Projectiles** and calculates ballistic trajectories towards the player, compelling frequent aerial combat engagements.

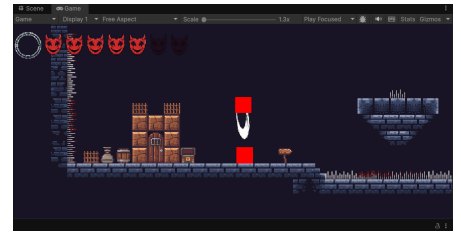
4) *Boss: Dark Fairy (Complex State Management)*: As the endgame challenge, the Boss AI focuses on rigorous **State Management**. The code strictly controls transitions between “Attack,” “Cooldown,” and “Movement” states. It features composite attack patterns (magic orbs, slashing, teleportation) and incorporates **“Anti-Close-Quarters Logic,”** automatically triggering a **Flee Behavior** when the player is detected within a critical proximity (See Fig. 4).



(a) Collider Configuration
(Hitbox vs. Hurtbox)

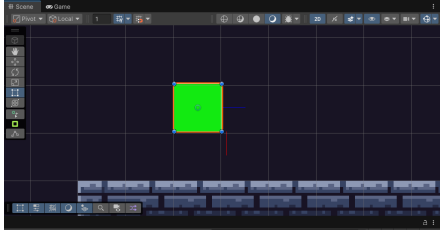


(b) Damage Knockback
(Vector Calculation)

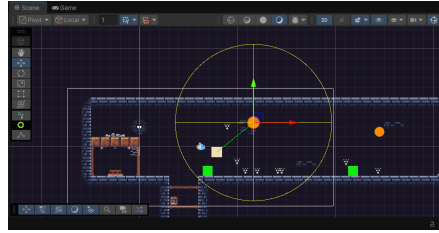


(c) Downward Strike (Pogo)
(Recoil Force Application)

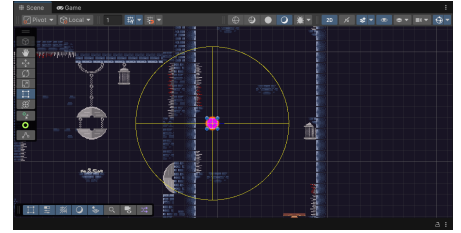
Fig. 2. Visualization of Player Physics Mechanics. (a) The strict separation of Hitbox (Red) and Hurtbox (Green) ensures precise collision detection. (b) Upon taking damage, a knockback vector is calculated to prevent stunlock. (c) The Pogo mechanism applies an upward impulse force when striking enemies from above.



(a) Ground Patroller
(Raycast Edge Detection)



(b) Self-Destructing Crawler
(Proximity Trigger Radius)



(c) Ranged Flying Unit
(Keep-Distance Logic)

Fig. 3. Visual Analysis of Enemy AI Behaviors. (a) Red lines indicate Raycasts detecting platform edges. (b) The wireframe sphere represents the detection range for triggering the self-destruct sequence. (c) The flying unit maintains a fixed vertical distance from the player.

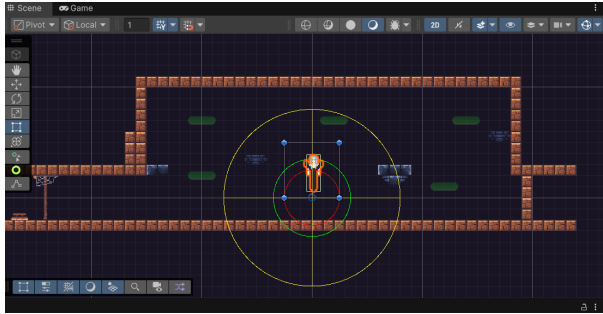


Fig. 4. Boss AI Spatial Logic. The visual gizmos illustrate the correlation between detection ranges and FSM state transitions. The outer ring triggers a "ranged attack" state, while the inner ring triggers "anti-melee" teleportation logic, and the outermost ring triggers a melee basic attack state.

C. Level Design and Environmental Interaction

To elevate the depth of map exploration and the technical threshold of platforming, the environment incorporates various physics-based interactive components and topographical mechanisms:

1) *Static Hazards (Physics-based Knockback)*: The map is densely populated with static threats such as Spikes and circular saws. Regarding collision logic, in addition to deducting Health Points, the system applies an opposing **Knockback Force**. This mechanism forcibly alters the player's **Rigidbody2D** velocity vector, effectively preventing the exploitation of Invincibility Frames to brute-force through obstacles, thereby imposing higher demands on operational precision.

2) *One-Way Platforms (Unidirectional Collision)*: Utilizes Unity's built-in **PlatformEffector2D** component to implement a unidirectional traversal topology. By defining the **Surface Arc**, the system permits characters to traverse the platform from below, triggering collision detection only when descending onto the upper surface. This design optimizes the fluidity of vertical movement and adds structural stratification to the level design.

3) *Dynamic Drop Traps (State Transition)*: Engineered dynamic traps featuring **State Transitions**. Initially in a static state, upon detection of player contact via **OnCollisionEnter2D**, the script switches the **RigidbodyType** to **Dynamic** and initiates gravity simulation, causing a delayed collapse. Crucially, these objects are registered within the **GameManager's** listener list. Upon player respawn, the system forcibly resets their **Position** and physical state, ensuring the **Replayability** of the level.

4) *Checkpoint System (State Persistence)*: Employs a "Bench" as the interactive interface for safe zones. Triggering interaction executes two core logic functions:

- **Spawn Point Update:**
Updates the global `respawnPoint` coordinate within the **GameManager**.
- **State Restoration:**
Resets player HP and Mana values to their maximums, achieving effective **Pacing Control** of the combat rhythm.

IV. IMPLEMENTATION DETAILS

The system architecture adheres strictly to the **Separation of Concerns (SoC)** principle, decoupling the framework into three core modules: Global State Management, AI Decision Making, and Physical Interaction. This section delves into the key algorithms and implementation specifics.

A. Global Management Architecture: GameManager

Serving as the central nervous system of the application, the GameManager orchestrates scene transitions, respawn logic, and victory determination. Its core design incorporates the following technical features:

1) *Singleton Pattern and Persistence*: The class is constructed using the **Singleton Pattern**, ensuring that a unique global management instance persists in memory throughout the lifecycle of object destruction and Scene Loading. Via the DontDestroyOnLoad mechanism, the system guarantees **State Persistence** across different scenes.

2) *Optimized Object Referencing and Robust Reset*: To mitigate **Performance Bottlenecks** caused by high-frequency calls to GameObject.Find within the Update loop, the system adopts **Caching** and **On-demand Retrieval** strategies:

- **Trap Reset**:

For DropTrap entities, the system maintains an object registry. Upon triggering RespawnPlayer(), it iterates through the list and invokes ResetTrap(), restoring their Kinematic/Dynamic rigidbody properties and spatial coordinates.

- **Deep Search for Inactive Boss**:

Addressing the “**Behavior Freeze**” anomaly where the Boss might be in an **Inactive** state (disabled) during respawn—rendering traditional Find commands ineffective—this implementation utilizes Unity’s advanced search API: FindFirstObjectByType (FindObjectsInactive.Include). This allows the manager to penetrate the active state constraints of the Scene Graph.

3) *Entity Logic Reactivation*: A critical synchronization issue was observed where enemy units, upon respawn, would render correctly but fail to execute behavior (the “Zombie State”). This occurred because the specific AI components (e.g., Patrol.cs, Kamikaze.cs) remained disabled even after the parent GameObject was reactivated.

To resolve this, we implemented an *Explicit Component Reactivation* step within the ResetEnemy() method. The system checks for the existence of specific AI scripts and forcibly sets their enabled property to true, ensuring the Finite State Machine (FSM) restarts its cycle immediately upon scene reset.

B. DarkFairyBoss: Finite State Machine (FSM) Architecture

The behavior of the Boss is not random but governed by a **Finite State Machine (FSM)** architecture. The Update() loop continuously monitors the isActing boolean flag and the distanceToPlayer value to serve as the decision basis for **State Transitions**.

1) *Coroutine-based Action Management*: All offensive maneuvers (Teleportation, Magic Orbs, Melee Slashing) are encapsulated within IEnumerator coroutines. This design allows the program to precisely control animation delays and cooldowns via yield return new WaitForSeconds() without blocking the **Main Thread**. For instance, the TeleportRoutine executes sequentially: Play Vanish Animation → Wait 0.5s → Modify transform.position → Play Appear Animation.

2) *Fail-safe and Reset Mechanism*: To resolve the “Boss Behavior Freeze” technical challenge mentioned in the abstract, a forced interruption logic was introduced within the ResetBoss() function:

```
public void ResetBoss() {
    StopAllCoroutines();
    isActing = false;
    anim.Rebind();
}
```

C. PlayerController: Physics-based Kinematics Implementation

The core of player manipulation lies in the precise interaction with the Unity Physics engine, rather than direct modification of Transform coordinates.

1) *Robust Ground Detection*: To achieve precise jump evaluation, this study discarded the unstable OnCollision event detection in favor of the Physics2D.OverlapCircle API. The system projects a microscopic circular detection area at the base of the character and combines it with a LayerMask to evaluate only the “Ground” layer. This method effectively prevents the “Infinite Mid-air Jump” bug caused by edge misjudgment.

```
// Physics-based Ground Detection
private bool CheckGround() {
    // Project a circle at the character's feet
    return Physics2D.OverlapCircle(
        groundCheck.position, // Center point
        checkRadius,
        // Detection radius (e.g., 0.2f)
        whatIsGround
        // LayerMask for ground objects
    );
}
```

2) *Invincible Dash Implementation*: The dash mechanism integrates Coroutines with the **Layer Collision Matrix**. Upon triggering a dash, the program temporarily moves the player object to the Ignore Raycast layer (or disables the Hurtbox) and applies a high instantaneous velocity vector. After a 0.2-second delay, the system automatically rolls back the state, realizing precise “**i-frames**” logic.

D. Enemy AI: Polymorphism and Vector Mathematics

The behavioral logic of hostile units demonstrates the application of **Polymorphism** and vector mathematics in object-oriented design.

1) *Vector-based Tracking*: The Self-Destructing Crawler AI utilizes vector subtraction ($\text{player.position} - \text{transform.position}$) to calculate the tracking direction vector and employs Vector2.Distance to monitor the target proximity in real-time. When $\text{distance} < \text{attackRange}$, an irreversible detonation sequence is triggered.

2) *Physics Constraints*: Addressing the common “Tumbling” issue in the Unity 2D physics engine, all ground-based enemies are forcibly configured during the initialization phase:

```
Rigidbody2D.constraints =  
RigidbodyConstraints2D.FreezeRotation
```

This command locks the Z-axis rotational degree of freedom, ensuring the character maintains an upright posture when subjected to forces or collisions.

V. EXPERIMENTS AND SYSTEM VALIDATION

This section presents the empirical validation results of the game system, focusing on verifying the **Correctness** of the AI behavioral logic, the **Stability** of the physics system, and the **Robustness** of the respawn mechanisms.

A. Simulation Environment

The project was developed and tested within the Unity 6 (2023 LTS) engine environment. The hardware testing platform operated on Windows 10/11 OS, with the display resolution set to a standard 1920×1080 . To ensure the universality of the platforming mechanics and the reproducibility of physics simulations, the gravity parameter of the physics engine was configured to the standard value: $\text{Physics2D.gravity} = \text{new Vector2}(0, -9.81)$.

B. Case Study: Resolution of State Deadlock

During the iterative development process, a severe **State Synchronization Anomaly** was observed: when a player died and respawned while the Boss was executing a “Casting” or “Teleporting” Coroutine, the Boss object would remain permanently frozen in a locked state where $\text{isActing} = \text{true}$.

1) *Problem Identification*: Due to the asynchronous nature of Coroutines, if the logic is not explicitly terminated before the object is disabled, this **Logic Lock** causes the AI to enter a Deadlock, rendering it unable to execute subsequent attacks.

2) *Solution and Verification*: This study introduced a forced interruption mechanism within the $\text{ResetBoss}()$ function:

- **Coroutine Termination**:

Invokes $\text{StopAllCoroutines}()$ to clear all scheduled attack commands.

- **State Reset**:

Forcibly resets the isActing flag to false and calls $\text{anim.Rebind}()$ to zero out the animator state machine.

- **Deep Retrieval**:

To address scenarios where the Boss might be in an inactive state, $\text{FindFirstObjectByType}(\text{FindObjectsInactive.Include})$ is employed

to ensure the reset command penetrates the hierarchy to reach the target.

Stress testing validated that the corrected Boss immediately resumes the Idle state upon player respawn and successfully restarts its player detection logic.

C. Verification of New Enemy AI

State transition logic validation was conducted for the newly introduced hostile units:

1) *Proximity-based Detonation (Crawler)*: Tests indicate that when the player enters the **Detection Radius**, the enemy correctly executes the state transition (Idle $\rightarrow$ Chase). When $\text{Vector2.Distance} < 1.0f$, the irreversible flashing warning and Area of Effect (AoE) Damage judgment are successfully triggered.

2) *Height-Locking Logic (Flying Unit)*: The stability of the hovering algorithm was verified. The unit consistently maintains a **Fixed Altitude** relative to the player, and the Instantiate method generates projectiles that accurately calculate vectors to track the player’s current coordinates.

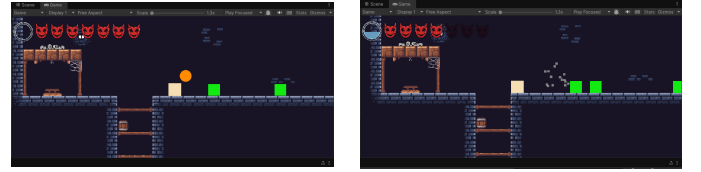


Fig. 5. Crawler Detonation. The AI triggers the explosion sequence upon detecting the player within the alert range ($< 1.0f$).

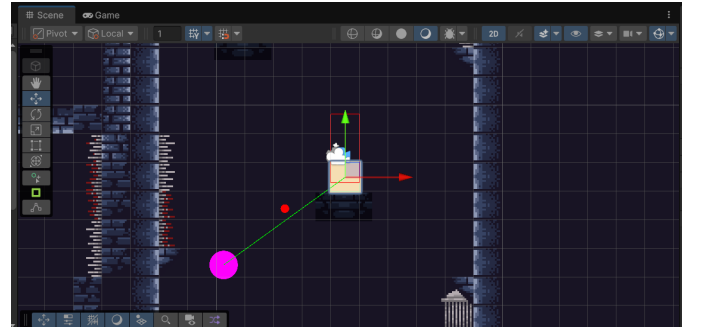


Fig. 6. Flying Unit Logic. The unit maintains relative altitude and fires projectiles using vector tracking.

D. Game Loop and Victory Condition

The final test verified the **Closed Loop** of the complete game cycle. When the Boss’s health reaches zero, the system sequentially triggers the following logic:

- 1) **Entity Despawn**: The Boss plays the death animation and executes $\text{Destroy}()$ or $\text{SetActive}(\text{false})$.
- 2) **System Freeze**: The GameManager successfully executes $\text{Time.timeScale} = 0.0f$, freezing all rigid-body physics calculations and animation frames.
- 3) **UI Feedback**: The Victory Panel pops up correctly, marking the termination of a single game cycle.

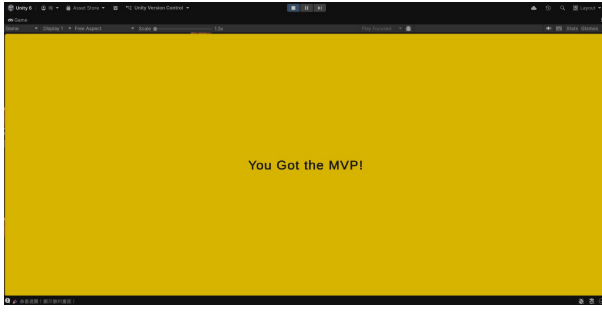


Fig. 7. The Victory Screen triggered after defeating the Boss

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

This project successfully realized a 2D action platformer utilizing the Unity engine and C#, validating the efficacy of **Object-Oriented Programming (OOP)** and **Finite State Machine (FSM)** principles in constructing complex interactive systems. The key contributions and findings of this research are as follows:

- **Robust Game Loop Architecture:** Through rigorous “Liveness-First” validation and input distribution logic, the system established a Game Loop characterized by high **Determinism**, effectively preventing logical dead-locks.
- **Resolution of Lifecycle Conflicts:** The development process underscored the critical importance of Object Lifecycle Management in Unity. This study proposed an asynchronous reset mechanism integrating the **Singleton Pattern** with **Coroutines**. This approach successfully resolved the technical challenge of “Boss Respawn State Synchronization,” eliminating the “Behavior Freeze” anomaly and providing a robust solution reference for the development of analogous game systems.
- **Physics-driven Gameplay:** By integrating Raycasting and vector mathematics into the physical control system, the project achieved a precise and fluid platforming experience.

B. Future Work

Based on the current system architecture, future work will focus on the following functional expansions and optimizations:

- **Advanced Audio Manager:** The current system lacks auditory feedback. Future iterations plan to introduce a global **Audio Manager** to implement dynamic fading for Background Music (BGM) and Spatial Audio processing for Sound Effects (SFX), thereby enhancing immersion.
- **Scalable Level Design:** Expanding the map system to support Multi-stage Levels and **Scene Streaming** technologies, alongside the introduction of non-linear exploration paths.
- **AI Behavior Trees:** As the variety of hostile units increases, the maintenance cost of FSMs rises significantly. Future research will investigate **Behavior Tree**

architectures to realize enemy AI with greater flexibility and decision-making depth.

REFERENCES

- [1] IncolGames, “Dungeon Platformer Tile Set (Pixel Art),” *itch.io*, Online. Available at: <https://incolgames.itch.io/dungeon-platformer-tile-set-pixel-art>.
- [2] PidrouDays, “Dark Fairy (Pixel Art),” *itch.io*, Online. Available at: <https://pidroudays.itch.io/dark-fairy-pixel-art>.